

Scanner Implementation Comparison: Manual vs JFlex

22I-0893 and 22I-1409

February 17, 2026

1 Overview

This document presents a comparison between two lexical analyzer implementations for the **CustomLang** programming language:

1. **Manual Scanner:** A hand-coded Java implementation using DFA logic.
2. **JFlex Scanner:** A generated scanner using the JFlex tool and regex specifications.

Both implementations were tested against a suite of 5 test files covering valid tokens, complex expressions, edge cases, partial implementations, and error handling.

2 Implementation Differences

Feature	Manual Scanner	JFlex Scanner
Architecture	Custom State Machine (DFA)	Table-driven Lexer (Generated)
Token Recognition	Explicit <code>if-else</code> and <code>while</code> loops	Regular Expressions in <code>.flex</code> file
Priority	Implemented via code execution order	Implemented via rule order in spec
Error Detection	Custom checks (e.g., <code>scanNumber</code> validation)	Catch-all rules and specific error regexes

Table 1: Functional Comparison

3 Output Comparison & Analysis

Both scanners produce identical token streams for valid inputs. However, subtle differences exist in error reporting strategies, particularly for boundary cases.

3.1 Case 1: Invalid Identifiers (e.g., "2Count")

Input: 2Count

Manual Scanner Behavior: The manual scanner follows a strict "longest match" for specific types. It consumes 2 as an `INTEGER_LITERAL`, stops at C, and then consumes Count as a separate `IDENTIFIER`.

- Output: `<INTEGER_LITERAL, "2">, <IDENTIFIER, "Count">`

JFlex Scanner Behavior: The JFlex specification includes a dedicated error rule for invalid identifiers starting with digits. It matches the entire 2Count sequence as an error.

- Output: `ERROR: Invalid Identifier (must start with uppercase)`

Conclusion: The JFlex approach provides more semantic error reporting for this specific case, while the Manual approach adheres strictly to token separation.

3.2 Case 2: Malformed Floats (e.g., "1.2.3")

Input: 1.2.3

Manual Scanner Behavior: Detects the second decimal point during the number scanning phase and immediately reports an error for the specific lexeme.

- Output: `ERROR: Malformed Literal: Multiple decimal points`

JFlex Scanner Behavior: Depending on rule definitions, JFlex may match 1.2 as a valid float and then .3 as a sequence of invalid characters (or a float missing an integer part), triggering the corresponding error rule.

- Output: `FLOAT(1.2), ERROR(.3)`

4 Side-by-Side Output Demonstration

The following table demonstrates the identical token stream produced by both scanners for the input: Count = 42; (Note: = is invalid in this version).

Manual Scanner Output	JFlex Scanner Output
<code><IDENTIFIER, "Count", Line: 1, Col: 1></code>	<code><IDENTIFIER, "Count", Line: 1, Col: 1></code>
<code>[Error] Line: 1, Col: 7, Lexeme: "="</code>	<code>[Error] Line: 1, Col: 7, Lexeme: "="</code>
<code><INTEGER, "42", Line: 1, Col: 9></code>	<code><INTEGER, "42", Line: 1, Col: 9></code>
<code><SEMICOLON, ";", Line: 1, Col: 11></code>	<code><SEMICOLON, ";", Line: 1, Col: 11></code>

Table 2: Output Comparison for Test 1

5 Performance Analysis

- **Manual Scanner:** Incurs overhead from method calls (`peek`, `advance`) and object creation during the scanning loop. Complexity is roughly $O(N)$.
- **JFlex Scanner:** Uses a pre-compiled transition table (DFA). This typically offers better raw performance ($O(N)$ with smaller constant factors) as state transitions are array lookups rather than conditional logic.

6 Conclusion

Both scanners successfully meet the language specifications. The **JFlex** implementation is stricter and more declarative, making it easier to maintain complex regex rules. The **Manual** implementation offers finer-grained control over execution flow and error recovery but requires more verbose code.